Laboratory 3: Graph Neural Networks

Economic Trade Network Analysis with a Graph Autoencoder

Course: CSCI 114 — Pattern Recognition | Work mode: Individual | Language: Python / PyTorch

Submission: lab3_gnn.py and lab3_answers.md | Deadline: November 6, 2026

Learning outcomes

1. Represent a relational dataset as a feature matrix X and undirected adjacency matrix A.
2. Implement mean-neighbor aggregation, message passing, and a two-layer graph encoder using matrix operations.
3. Train a graph autoencoder and evaluate candidate links, embedding similarity, and anomaly scores.
4. Interpret changes in learned representations under a controlled perturbation.
5. Explain limitations of graph reconstruction and distinguish learned associations from real-world forecasts.

Problem scenario

Eight fictional countries have five economic indicators and an undirected network of significant trading relationships. Build a graph neural network (GNN) to learn three-dimensional representations of the countries. Use these embeddings to investigate possible trade links, similarity, unusual network positions, and a hypothetical inflation shock. These data are synthetic; economic findings are illustrative rather than policy predictions.

Requirements and restrictions

- Use Python 3 and standard PyTorch (torch, torch.nn, torch.optim). No PyTorch Geometric, DGL, or prebuilt NetworkX GNN implementations.
- Implement aggregation yourself using PyTorch matrix operations; preserve an unmodified full graph separately from the training graph.
- Set torch.manual_seed(42) before initializing each compared model. Report environment and package versions.
- Submit runnable source code and a Markdown report with all guide-question responses, actual numerical output, and discussion.

Part A — Dataset and graph exploration

A1. Economic features

Create X ∈ R^(8×5) in the country order shown. Columns are GDP growth (%), inflation (%), unemployment (%), exports (% of GDP), and FDI (% of GDP).

Index	Country	GDP growth	Inflation	Unemployment	Exports	FDI
0	Aridia	5.8	3.1	4.2	46	6.2
1	Borland	5.1	3.8	5.0	42	5.7
2	Cyrenia	6.4	2.7	3.8	51	7.1
3	Deltora	3.2	6.5	7.4	30	3.1
4	Estovia	2.8	7.2	8.1	27	2.8
5	Freland	4.4	4.1	5.6	39	4.9

6	Galenia	5.6	3.0	4.5	48	6.5
7	Hesperia	3.5	5.8	6.9	33	3.6

A2. Trade network

Build an 8×8 binary, symmetric adjacency matrix A_full using the following 13 undirected edges. Do not add self-loops to A_full.

Aridia–Borland; Aridia–Cyrenia; Aridia–Galenia; Borland–Freland; Borland–Galenia; Cyrenia–Freland; Cyrenia–Galenia; Deltora–Estovia; Deltora–Freland; Deltora–Hesperia; Estovia–Hesperia; Freland–Galenia; Freland–Hesperia.

Verify A_full = A_full.T and diag(A_full) = 0. Compute each country's degree, the mean degree, and the number of unique edges. Explain why summing all entries of a symmetric adjacency matrix double-counts undirected edges.

A3. Standardization

Compute X_mean = X.mean(dim=0), X_std = X.std(dim=0), and X_scaled = (X − X_mean) / X_std. Retain the original training mean and standard deviation for later perturbation experiments. State why feature scales matter.

Part B — Build and train a graph autoencoder

B1. Withheld links and training graph

Copy A_full to A_train and remove BOTH directions of Aridia–Galenia (0,6) and Deltora–Hesperia (3,7). Use A_train, never A_full, for encoder message passing. Verify the training graph contains 11 unique edges.

B2. Mean aggregation function

Write aggregate(A, H). Internally compute Â = A + I, the row-degree matrix D of Â, and D^(−1)ÂH. Use the same function for both GNN layers. Handle degrees safely and check returned dimensions.

B3. Encoder and decoder

Define GNNEncoder(nn.Module) with aggregate → Linear(5,8) → ReLU → aggregate → Linear(8,3). It must return embeddings Z with shape [8,3]. Define GraphAutoencoder(nn.Module) with reconstructed adjacency probabilities sigmoid(ZZ^T), returning (A_reconstructed, Z).

B4. Training configuration

Setting	Value
Random seed	42
Hidden dimension	8
Embedding dimension	3
Optimizer	Adam
Learning rate	0.01
Epochs	2000
Loss	nn.BCELoss()
Input features	X_scaled
Encoder graph	A_train
Baseline reconstruction target	A_train

Train the source exercise's baseline, printing loss every 200 epochs. Print the final loss, all eight embeddings, and the reconstructed 8×8 probability matrix. The matrix is a score matrix, not a ground-truth adjacency matrix.

Methodological caution: because A_train sets withheld positive links to zero, ordinary BCE penalizes positive predictions on those links during training. Therefore the test is a challenging diagnostic, not a strictly clean holdout. Keep this baseline for comparability; discuss the issue in your report. An optional extension is to mask the withheld entries from BCE and compare results.

Part C — Pattern recognition using learned embeddings

C1. Hidden-link comparisons

Report sigmoid(Z[i]·Z[j]) for Aridia–Galenia versus Aridia–Estovia, and Deltora–Hesperia versus Deltora–Cyrenia. Indicate whether each hidden real link outranks the corresponding non-edge. Do not treat scores as calibrated economic probabilities.

C2. Candidate ranking

Enumerate all unordered pairs (i,j) with i < j and A_train[i,j] = 0, ignoring diagonal entries. Sort by reconstructed score, print the top five, and report the ranks of both withheld real edges. Use a deterministic tie-breaking rule, e.g., by pair index.

C3. Country similarity and anomaly detection

For each country, find its nearest OTHER country by Euclidean distance in the three-dimensional embedding space. Display country, nearest neighbor, and distance. Then compute the embedding centroid, calculate each node's Euclidean distance to that centroid, and rank descending as a simple anomaly heuristic. Explain why this does not establish real economic abnormality.

Part D — Shock propagation and baseline comparison

D1. Controlled inflation shock

Copy original X, change ONLY Borland's inflation from 3.8 to 12.0, and standardize with the SAVED original X_mean and X_std. Without retraining, use the trained encoder with A_train to obtain Z_after. Compare $||Z_after[i] - Z_before[i]||_2$ for Borland, Aridia, Freland, Galenia, and Estovia. Discuss direct neighbors, two-hop reachability, and observed embedding changes. This illustrates message passing, not macroeconomic causation.

D2. Non-graph baseline

Implement BaselineEncoder with Linear(5,8) → ReLU → Linear(8,3), receiving X_scaled only (no adjacency matrix). Explain what the GNN has that this feature-only encoder lacks. Comparing numerical embeddings meaningfully requires a declared training objective; merely instantiating an untrained baseline does not provide a valid performance comparison.

Part E — Optional advanced extension

Replace row-mean aggregation D^(−1)Â with symmetric normalization D^(−1/2)ÂD^(−1/2). Reinitialize with seed 42, train using the same settings, and compare final loss, withheld-link scores/ranks, nearest neighbors, and anomaly ordering. Discuss why lower reconstruction loss need not imply better hidden-link rankings. Optional further investigation: exclude both withheld edge positions from reconstruction loss and compare with the original baseline.

Required program output

- Matrix/graph verification: X and A shapes, symmetry and diagonals, degrees, average degree, unique edge count.
- Training: loss every 200 epochs, final loss, embeddings (8×3), reconstructed adjacency matrix (8×8).
- Link analysis: four pair scores, top five candidates, ranks of both withheld links.
- Representation analysis: nearest-neighbor table, anomaly ranking, and shock-induced changes for all five named countries.

Summary of guide questions

Answer all questions in lab3_answers.md. Cite computed values, rankings, or specific country pairs where requested. Use clear numbered answers corresponding to GQ1–GQ16.

6. GQ1. Which country has the largest degree, which has the smallest, what is the mean degree, and how many unique edges exist? Why is summing A_full misleading?
7. GQ2. Give two economic reasons why information from trading partners may improve a country representation beyond its own five indicators.
8. GQ3. Why is per-feature standardization needed? Explain the effect of different numeric ranges.
9. GQ4. Explain the role of self-loops and D^(−1) in mean aggregation. What does a single aggregation step compute?
10. GQ5. How many hops can influence a node after one and two GNN layers? Identify a two-hop path from Freland to Aridia in A_train.
11. GQ6. Why can learned GNN embeddings capture information unavailable in the original feature vectors?
12. GQ7. Report and compare both hidden-edge scores against their specified non-edge controls. Did either real edge score higher?
13. GQ8. Give the top five candidate missing edges and the rank of each withheld edge. Would either have been discovered near the top?
14. GQ9. Do nearest-neighbor embeddings reflect features alone or network topology as well? Support with a named pair.
15. GQ10. Which country has the largest centroid-distance anomaly score? Relate it to original indicators and neighboring countries; state a limitation.
16. GQ11. Quantify the inflation shock's effect on Borland and the four named countries. Which changed most, and why can non-neighbors change?
17. GQ12. Contrast the available information in X, A, the GNN, and the feature-only baseline.
18. GQ13. What does the BCE target A_train imply for the two withheld real links? Why does this limit a clean held-out evaluation?
19. GQ14. List at least three reasons model scores cannot be interpreted as forecasts of future trade.
20. GQ15. Discuss one technical or evaluation limitation of this eight-node experiment and one way to address it.
21. GQ16. Summarize your most important finding with numerical evidence. If completing the optional extension, explain whether symmetric normalization improved hidden-edge ranking.

Assessment rubric — 100 points

Criterion	Points	Evidence expected
Data preparation and graph correctness	15	Accurate X, A_full, A_train; symmetry; degrees and standardization

Aggregation, encoder and decoder	25	Correct matrix aggregation, two GNN layers, shapes, BCE reconstruction
Training and reproducibility	10	Specified hyperparameters, seed, readable outputs, successful run
Link prediction and embedding analysis	20	Pair scores, candidates/ranks, similarity and anomaly interpretation
Shock experiment and baseline	10	Correct reuse of scaler/model, quantitative changes, conceptual baseline
Guide-question answers and critical reasoning	15	Complete evidence-based answers, explicit holdout caveat, limitations
Code quality and submission completeness	5	Organization, comments, naming, both required files

Total: 100 points. Optional advanced work may be recognized at the instructor's discretion; it does not replace required work. Results are graded for methodological correctness and interpretation, not for achieving predetermined hidden-edge scores.

Submission checklist

Before uploading, confirm every item below:

☐ lab3_gnn.py runs from a clean start using ordinary PyTorch; no disallowed GNN package is used.

☐ lab3_answers.md includes numbered answers GQ1–GQ16.

☐ Source code defines the aggregate function, GNNEncoder, GraphAutoencoder, and BaselineEncoder.

☐ X has shape [8,5]; A_full and A_train have shape [8,8]; Z has shape [8,3].

☐ Two withheld links are removed symmetrically from A_train, while A_full remains unchanged.

☐ Console output includes degrees, training loss, all embeddings, and reconstructed adjacency probabilities.

☐ Console output includes both hidden-edge comparisons, top five candidate links, and both hidden-link ranks.

☐ Console output includes nearest-neighbor results, anomaly ranking, and Borland shock comparisons.

☐ Written answers include numerical evidence, limitations, and a clear explanation of the withheld-link training issue.

☐ Code is readable, seeded, and provides any necessary run instructions or dependency notes.

Submission format: Upload the two files lab3_gnn.py and lab3_answers.md (or a single ZIP containing both, if instructed). Do not submit screenshots in place of runnable code. Include optional extension findings in the Markdown report if attempted.